

// Day 22 of 30

GUIDE – 5 STRATEGIES

Redis Caching Strategies for Production

→ Swipe through all 6 slides



Shubham Bhati

[linkedin.com/in/bhatishubham](https://www.linkedin.com/in/bhatishubham)

1/8

Why Cache?

Same DB query 1000 times/minute = waste

Without cache:

Every request → DB query → 50ms
1000 req/min → 1000 queries/min

With Redis cache:

First request → DB → cache → 50ms
Next 999 → cache → 1ms each

- 50x faster for repeat reads
- 99.9% reduction in DB load

💡 Cache what's read often, changes rarely



Shubham Bhati

[linkedin.com/in/bhatishubham](https://www.linkedin.com/in/bhatishubham)

Strategy 1: Cache-Aside

Most common. You control everything.

Read:

1. Check cache
2. HIT? → return from cache
3. MISS? → query DB
4. Store in cache
5. Return to client

Write:

1. Update database
2. Invalidate cache (delete key)

// Next read will refresh cache

💡 Best for: most read-heavy applications



Shubham Bhati

[linkedin.com/in/bhatishubham](https://www.linkedin.com/in/bhatishubham)

Strategy 2: Write-Through

Write to cache AND DB together

Write:

1. Write to cache
 2. Cache writes to DB
- Both always in sync

Read:

→ Always from cache (always fresh)

Pros: Cache always up-to-date

Cons: Write latency higher
(two writes per operation)

💡 Best for: data that must always be consistent



Shubham Bhati

[linkedin.com/in/bhatishubham](https://www.linkedin.com/in/bhatishubham)

Strategy 3: TTL-Based

Set expiry. Let cache auto-refresh.

```
SET user:123 '{ ... }' EX 300
```

- Data cached for 5 minutes
- After 5 min, key expires
- Next request fetches from DB
- Re-cached for 5 more minutes

Simple. Effective.

Acceptable staleness = TTL duration

💡 Best for: data that can be slightly stale (menus, profiles)



Spring Boot + Redis

4 lines to add caching

1. Add dependency:
`spring-boot-starter-data-redis`
2. Enable caching:
`@EnableCaching`
3. Cache a method:
`@Cacheable("users")`
`public User findById(Long id) { ... }`
4. Invalidate:
`@CacheEvict("users")`
`public void updateUser(User u) { ... }`

💡 Spring makes Redis caching trivially easy



Shubham Bhati

[linkedin.com/in/bhatishubham](https://www.linkedin.com/in/bhatishubham)

Common Pitfalls

Caching gone wrong

1. Cache stampede:
 - 1000 requests hit expired key
 - All query DB simultaneously
 - Fix: distributed lock on refresh
2. Stale data served:
 - User updates profile, sees old data
 - Fix: evict on write, not just TTL
3. Memory overflow:
 - Fix: set maxmemory + eviction policy

💡 Caching is easy. Caching CORRECTLY is hard.



done = true

Follow for more!

Like · Comment · Repost

Shubham Bhati

[linkedin.com/in/bhatishubham](https://www.linkedin.com/in/bhatishubham)



Shubham Bhati

[linkedin.com/in/bhatishubham](https://www.linkedin.com/in/bhatishubham)